

Chapter 10

System Development 2

- Testing and Documentation

Testing the Program

Traditional Desk Checking

1. Reading or Inspecting

- A programmer desk check the program via process of *reading* or *inspecting* the program, visually (traditional approach).
- **Disadvantage** : detecting errors is difficult.



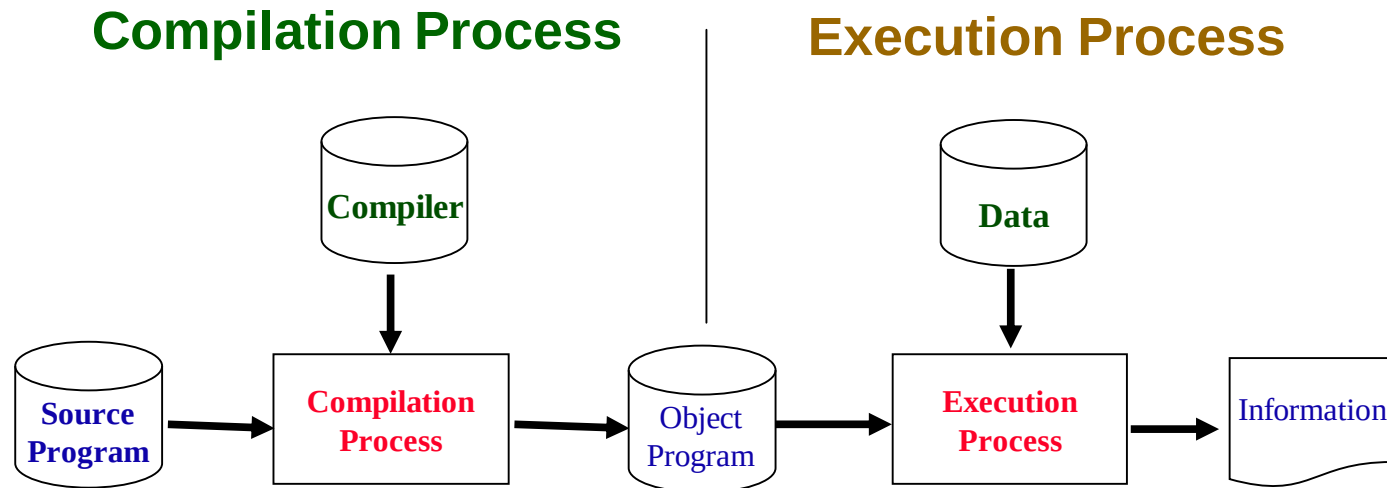
2. Structured Walkthrough

- The **developer** presents his/her module to a review team (other programmers, systems analyst, users, auditors).
- **Other participants** :), chairperson. **recorder** (secretary
- **Test Areas** : Detect errors, adherence to standards, and verify that the program satisfies the requirements.
- The developer would use the feedback to further improve the module design.

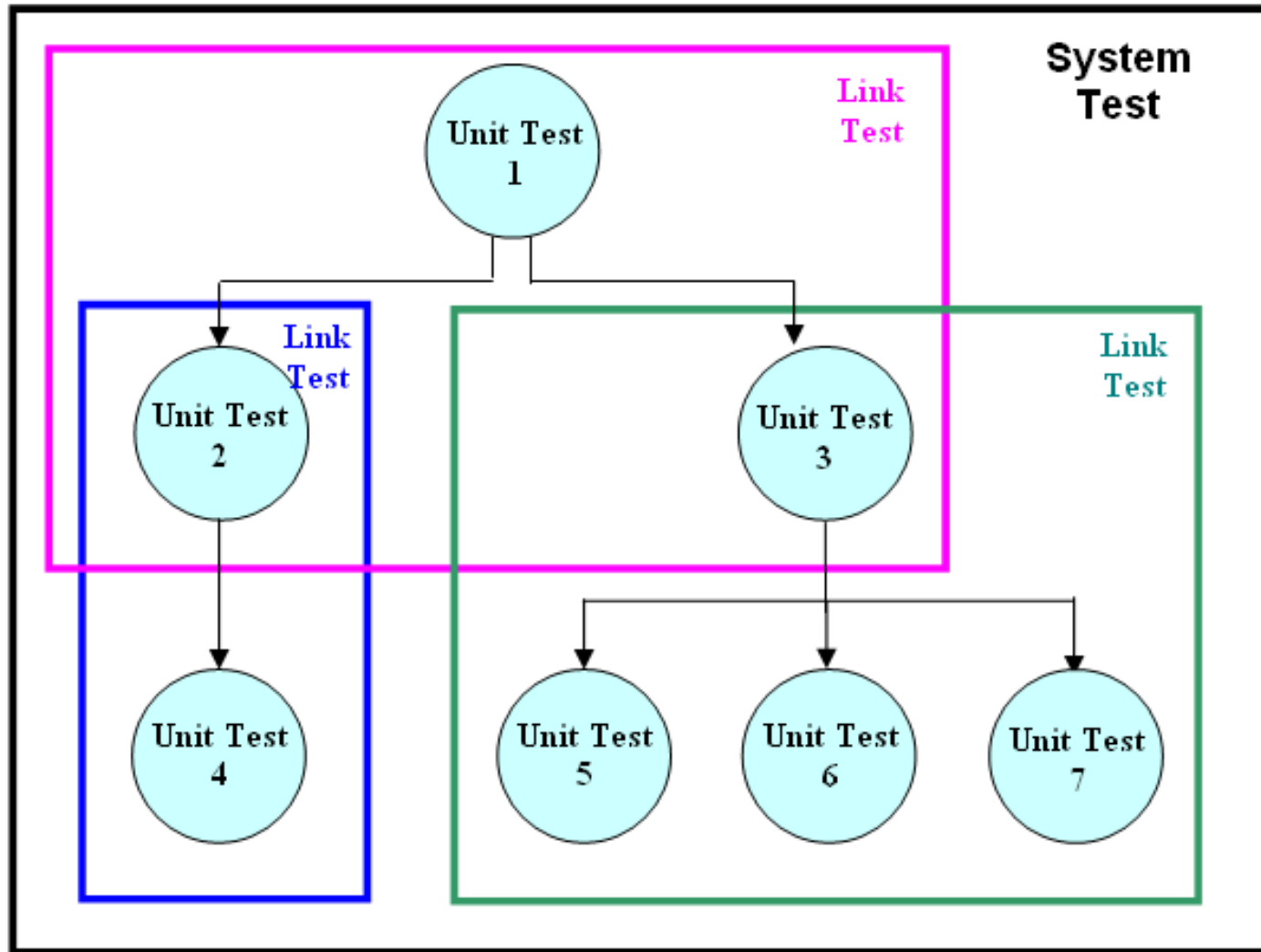


Program Translation

- **Purpose** : to translate the program into executable object codes.
- **Test Areas** : identify **syntax errors**, which are language violations caused by data entry mistakes, inconsistencies in the program, and language grammar errors.
- The programmer corrects the program to eliminate these syntax errors and re-submits for translation.



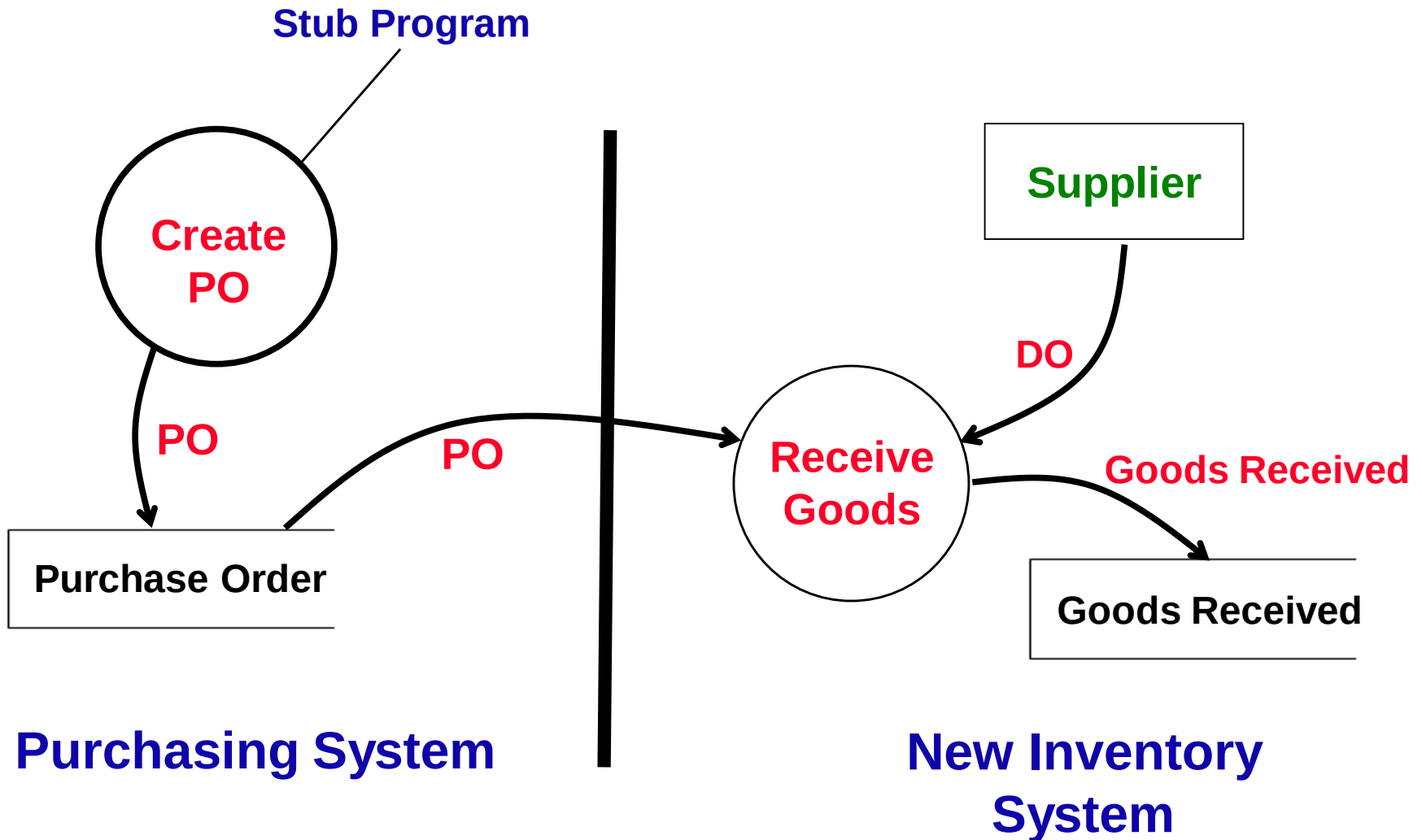
Overview of Testing Methods



1. Unit Testing

- **Number of programs.** To test **one** program individually and independently.
- **Purpose.** Ensure that each unit or module can **work properly** and in accordance with program specification. Identify and eliminate execution errors, logic errors, and syntax errors
- **Who perform.** **Programmers** testing the individual programs.
- **Test data.** Create **dummy** data.

Example of a Stub Program



Stub Testing

- **Stub** : A piece of codes written to **emulate** a module, to stand in for the missing module (eg Purchasing System).
- **Purpose** : Test to ensure that the program can interact (or **integrate**) with other program (a stub) (eg test *Inventory System* which interact with *Purchasing Stub*) and their files individually, before they are integrated into the system later.
- **Who perform** : **programmers**
- **Test data** : Create **dummy** data

2. Link / Integration Testing

- **Number of programs.** To test **two or more** related programs together as a group, individually and independently.
- **Purpose.** To ensure that the data **passed** between the two programs (book loan program and book return program) can be carried out correctly.
- **Who Perform.** **Programmers**
- **Test data.** Create **dummy** data.

3. System Testing

- **Number of programs.** All the programs in a system as a whole.
- **Purpose.** Ensure that the individual programs can work properly as an entire whole system.
- **Who Perform.** *End-users* (prepare source documents, enter data, and select reports for output.) *Operations group* (run the system in compliance with the operations documentation). *Project team* (ensure the system functions correctly).
- **Test data.** Mainly real data (samples from users).

Scope of System Testing

1. **Functionality.** Ensure that the functions defined in the analysis and design stages of development are correctly implemented in the software.
2. **Usability.** Ensure there are : **consistencies** (with other modules), adherence to **industry standards**, clarity and usefulness of **Help** facility and **error** messages.
3. **Interfaces.** Ensure that the system interfaces correctly with **other** systems (eg. passing data from one to the other –purchasing system and new inventory system).
4. **Stability and reliability.** Ensure that the system will run in stable manner providing consistent and correct results. The system must also run continuously in a reliable manner without any breakdowns for a long period of time.

4. User Acceptance Testing (UAT)

- **Number of programs.** Test **all** the programs (or modules) for the entire system, either individually or as a group for related programs.
- **Purpose.** Test whether the working system meets users' **requirements** as defined in the System Requirements Specification (SRS) (both functional and non-functional eg usability)
- **Who Perform.** **Users** test whether the system meets their requirements. Sometimes with assistance from programmers.
- **Test Data.** Mainly samples of **real world data**.

User Acceptance Testing (UAT)

..cont

- **Other Minor Concerns (incidentally)**
 - **Errors or bugs** which may have not detected during Testing stage.
 - **Requirements** (hardware and software) of the new system which may be overlooked.
 - **Operating procedures changes** may have not been taken into account.

Exercise for Students

Areas	Unit Test	UAT	System Test
No. of Programs			
Purpose	Ensure that each unit functions properly according with program spec.	Test whether the whole system meets the defined requirements in the Requirements Specification.	Ensure the individual programs can work properly as a whole system.
Who Perform			
Test Data Used			

Areas	Unit Test	Integration Test	User Acceptance Test	System Test
Number of modules	Testing of individual programs, separately and independently.	Testing of two or more programs which are connected together.	Testing of the entire system consisting of many programs	Testing of the entire system.
Purpose	Ensure that each unit / module functions properly & works in accordance with <i>program specification</i> .	Ensure that one program can pass data successfully to another program.	Ensure that the entire system meets the users' requirements as stated in the System Requirements Specification.	Ensure that the entire system consisting of many programs can work together as one whole system.
Who Perform	Programmers who have been assigned	Programmers of the related programs	End-users (sometimes with assistance from the programmers).	End-users, Development Team Operation group
Test Data	Dummy Data	Dummy Data	Dummy and Real Data	Real Data

Testing routes

Develop system -> Quality assurance (1 round of testing) - OK

SEND TO CLIENT

IT Team -> SIT (System Testing)

blindly test the system based on the document
- OK

UAT(User Acceptance Test) - REAL USERS - OK

SEND the system to Production

Types of Acceptance Testing

- **Alpha Test.** Test performed in-house on bespoke systems developed for a **single client** at software **developer's** premise. It attempts to simulate the real environment.
- **Beta Test.** This is carried out by selected **customers** who agree to use (test) that system at their **own** premises and provide feedback. This exposes the product to real use and detects errors.
- **Benchmark Test.** A set of test cases that represent typical conditions (for a standard system) under which the new system will be evaluated for their **performance**.
- **Parallel Test.** This is performed when the *new* and the *old* systems are run concurrently and their results **compared** for any unacceptable deviations.

Exercise for Students

Area	Alpha	Beta
Purpose	Validate the quality of the system using automated or manual testing scenarios	Get real world feedback from actual selected group of users
Carried out by	In house developer	Selected units/ selected group of users
Test on	Functionality and usability	usability, functionality, security, and reliability are tested to the same depth
Location	Within the organization	User's enviroment

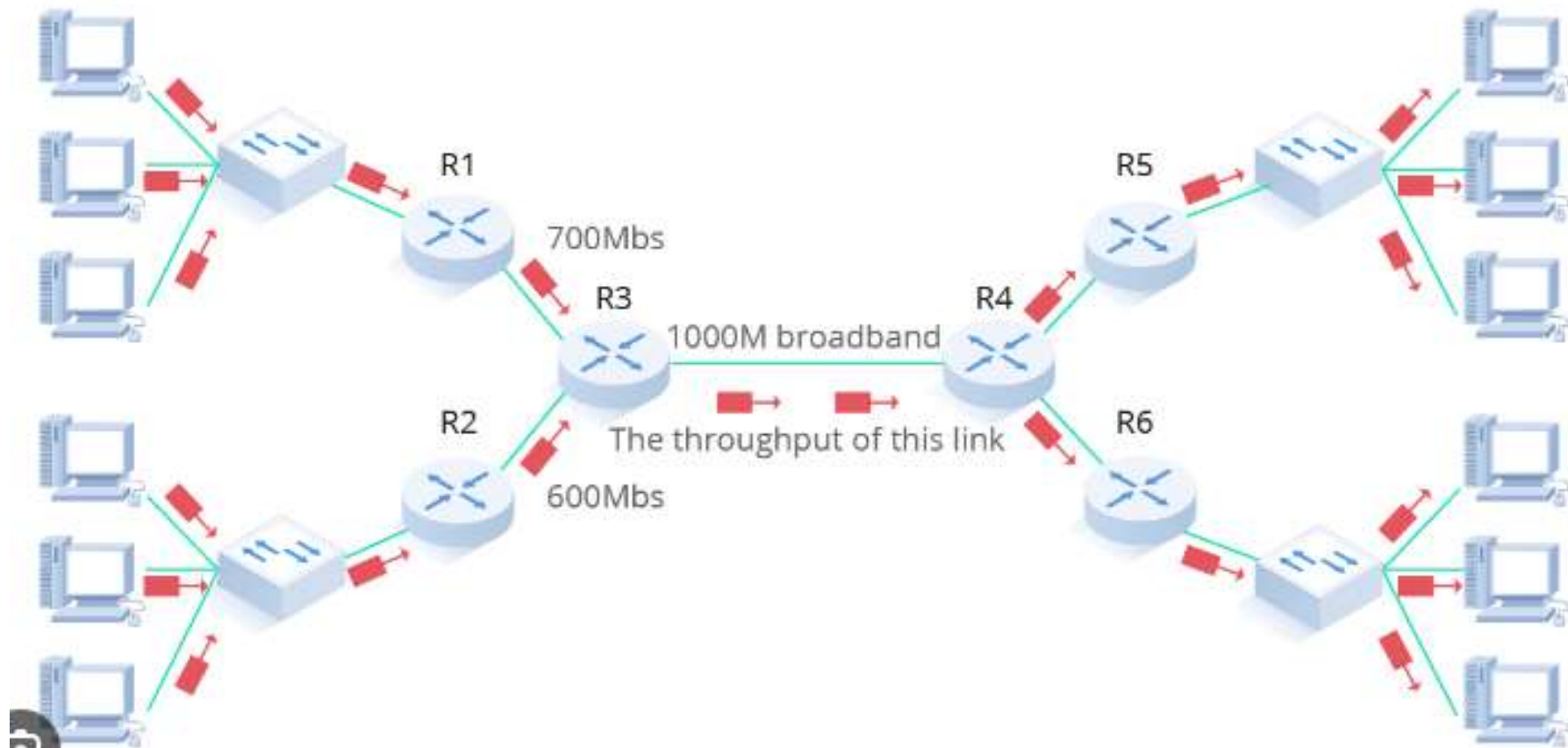
5. Load (Performance, Stress, Volume) Testing

- **Meaning**

- **Concerned.** Putting the system under large data conditions to explore the **performance** of a system.
- **Example.** Test for response time for specified **data volumes** (eg. 60,000 transactions), **network traffic**, **number of users** (eg 2 millions users).
- **Method.** Use automated testing tools to create the data conditions (users, traffic and input data)

- **Purpose / Role**

- Test whether the system can meet defined **performance objectives** (eg 2 second response time).
- Test whether the system can achieve this objective with the given or predicted **input data volumes**, **network traffic**, and **number of users**.



Volume Testing

- behavior of an application by inserting a massive volume of the load in terms of data
- You give a HUGE number of data (**60,000 transactions /input data**) and see how well the system performs.

Stress Testing

- To test the system by pushing the system to its limits, see what's the maximum stress and the maximum limit the system can handle
- You give a max number of data (max **70,000 transactions /input data**) and see how well the system performs.

Performance Testing

- To test the system by have it perform under many number of users (**eg 2 millions users**) to simulate as real as possible over a period of time

Uses/Benefits of Load Test

- **Measure.** Able to measure the **performance** of an application with current data characteristics.
- **Predict.** Helps to predict the **performance** of the system with future forecasted data characteristics.
- **Explore.** To explore the potential **effects** of various ***data volume***, ***users*** and ***data traffic***.

Examples of Load (Performance) Testing

- **Order Online** – test to determine the potential **performance** effects of an increase in the number of customers accessing the website at the same time.
- **Point Of Sales (POS) system / (cashier system)** – test to determine the **performance** effects of an increase in the volume of transactions at the check-out point/cashier counter.

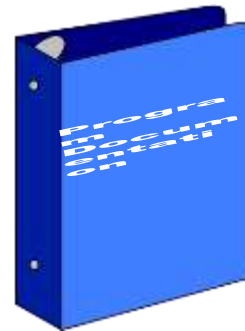
Automated Testing Tools

- **Importance of Testing Tools.** (a) testing affects **costs** (create data, execute testings), (b) affects **quality** (on time and within budget)
- **Examples.** Computer Aided Software Testing (CAST) tools. Selenium
- **Tasks. Include :**
 - create test data: input data volume, users and traffic
 - repeating test executions
 - assess performance
 - simulation of system interfaces (eg stubs / stimulated modules)

Benefits of Automated Testing Tools

1. **Save time and costs.** Create *data* (*users, data, traffic*) and execute testings.
2. **Predict performance.** Measure the performance of predicted data characteristics. Take preventive measures.
3. **Diagnoses.** Allow the tester to *identify* the bottlenecks and design *solutions* to solve the bottlenecks.

Documentation (4 types)



1. End User Documentation

- **Prepared by** : system analysts and programmers
- **Used by** : end users and trainers
- **Purpose** : Used to teach users how to use the software or self-training.
- **Content** :
 - **Input.** Describe step-by-step procedures for inputting data and control.
 - **Output.** Describe types and forms etc. Customer Registration Form
 - **File amendment procedures.** Explanation of authorisation and documentation requirement.
 - **Error reports.** Explanation of nature and form of error reports (eg. exception reports for items rejected by data validation checks) and action to be taken.

2. Operation Documentation

- **Prepared by** : systems analyst
- **Used by** : operations group
- **Purpose** : Assist the operations group in operating the system, schedules the execution of batch jobs, distributes printed outputs.
- **Content** :
 - Systems set-up procedures.
 - Security procedures eg. authorisation for use of systems.
 - File backup and recovery procedures in the event of a systems failure.
 - System error messages and responses to take.

3. Program Documentation

- **When** : Begins in System Analysis phase and completed after testing.
- **Prepared by** : System analysts (process descriptions, report layouts), programmers (eg. Comments with the programs)
- **Used by** : programmers
- **Purpose** : for maintenance or enhancement
- **Content** :
 - Includes *notes*, *flowcharts*, a listing of all the *codes*, and *tests* carried out, *test data* and *expected results*.

4. System Documentation



- **When** : Begins in System Analysis and completed in System Design
- **Prepared by** : system analyst / Business Analyst
- **Used by** : programmers, designers, maintenance programmers
- **Purpose** : System documentation describes the **features** of an information system and explains how these features are **implemented**.
- **Content** :
 - Data dictionary entries, DFDs, screen layouts, source documents, and the systems request form.

Online Documentation

- Most documentation now are typically in the form of online help. Some examples are shown below.

Table 15-6

Outline of a Generic User's Guide

Preface

1. Introduction

1.1 Configurations

1.2 Function flow

2. User interface

2.1 Display screens

2.2 Command types

3. Getting started

3.1 Login

3.2 Logout

3.3 Save

3.4 Error recovery

3.n [Basic procedure name]

n. [Task name]

Appendix A—Error Messages

([Appendix])

Glossary

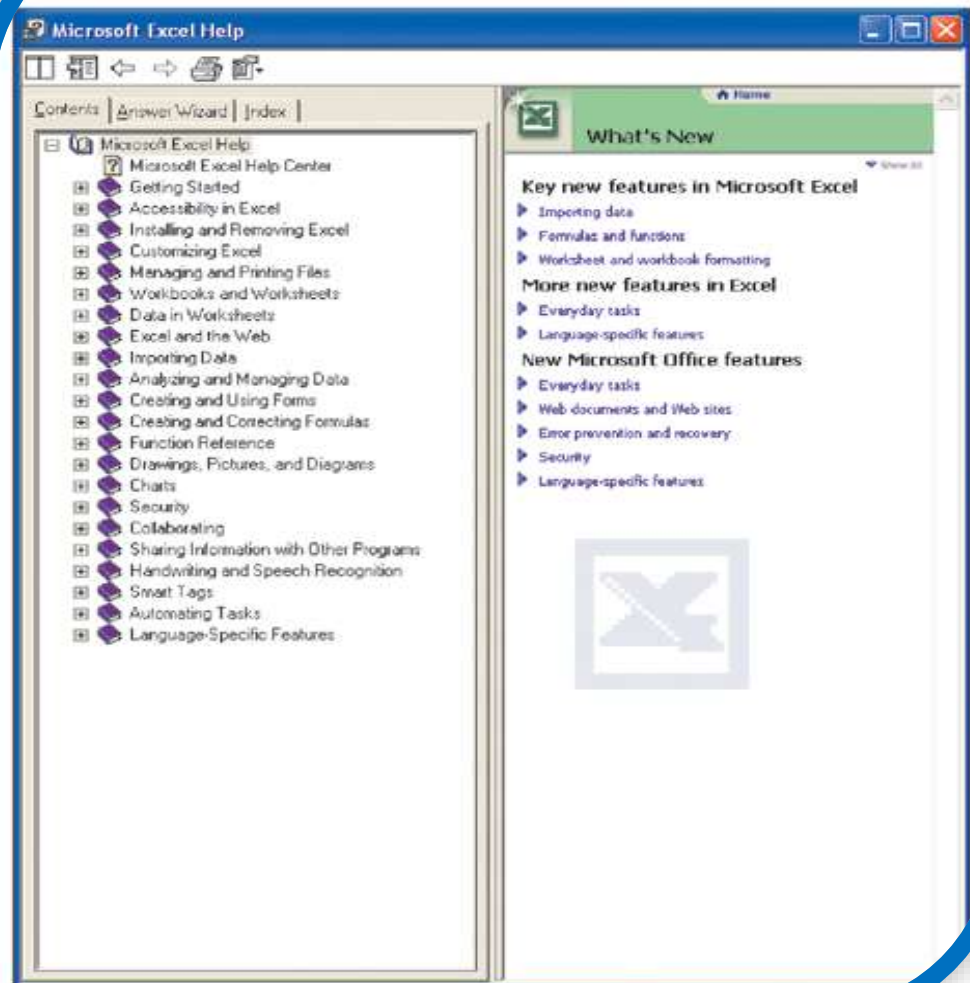
Terms

Acronyms

Index

(Source: Adapted from Bell and Evans, 1989.)

Figure 15-8 Contents of the help facility in Microsoft Excel™



Exercise for Students

Differences	System Doc	Program Doc	User Doc	Operation Doc
Prepared by				
Used by				
Purpose				
Content				

Suggested Answers

Differences	System Doc	Program Doc	User Doc	Operation Doc
Prepared by	System Analyst	Programmers	Analyst & Users	System Analyst
Used by	Designers, programmers, maintenance programmers	Programmers	Users, trainer	Operation staff
Purpose	Implementation , maintenance	Maintenance	Training, learning	Operation
Content	Data dictionary, screen layouts, ERD, DFD	Flowcharts, codes	Input, output procedures	Back-up and recovery procedures